Playwright BDD & IA

Architecture expliquée • Définitions • Parcours guidé • Préparation à l'entretien

UI_PLAYWRIGHT_BDD

Des tests exécutables.
Des preuves observables.
Une assistance IA contrôlée.

Socle de test

Cucumber interprète les scénarios Gherkin ; Playwright pilote le navigateur.

Les assertions établissent le résultat du test.

Assistance IA

Les agents aident à préparer les tests, analyser les échecs et formuler des recommandations.

Le modèle peut proposer une réponse incorrecte.

LLM harness

Infrastructure qui encadre le modèle : contexte, appels, contrôles et suivi.

Ce terme regroupe plusieurs modules existants.

Périmètre : **ui_playwright_bdd uniquement**. État décrit d'après le code et les documents locaux consultés le 29 septembre 2026. La présence du code ne prouve pas la disponibilité actuelle des services externes.

Deux chaînes complémentaires

La chaîne de tests produit des résultats. La chaîne IA aide à les préparer et à les interpréter.

Chaîne fonctionnelle

Exigences
Jira et critères d'acceptation

→

Features
Scénarios Gherkin revus

→

Steps & pages
Code TypeScript et Page Objects

→

Exécution
Cucumber + Playwright

→

Preuves
Résultats, traces et Allure

Chaîne d'assistance

Agents QA
Génération, analyse, recommandation

→

LLM harness
Prépare et contrôle les échanges

→

Groq / Ollama
Fournisseur cloud ou moteur local

→

Vérificateurs
Contrôles des sorties et références

→

Revue QA
Décision fondée sur les preuves

Avant les tests

Planning examine la couverture. Codegen propose des scénarios et du code.

La revue métier reste nécessaire.

Après les tests

Quality et Bug exploitent les échecs ; Advisor formule une recommandation.

Une hypothèse de cause doit être vérifiée.

Pilotage

Pipeline ordonne les commandes et suit leurs états.

Tous les modules ne s'exécutent pas à chaque commande.

> Le diagramme exprime des responsabilités. Il ne signifie pas qu'une revue humaine ou qu'un appel IA est automatiquement déclenché à chaque flèche.

Du langage métier à la preuve

Exemple pédagogique : vérifier qu'une tâche créée est visible dans la liste.

Feature / scénario

Un fichier .feature décrit un comportement attendu avec Given, When, Then.

Exemple : une tâche saisie doit apparaître.

`src/features/`

Step definition

Une fonction TypeScript associée à une phrase Gherkin.

Elle orchestre les actions et vérifications.

`src/steps/`

Page Object

Classe qui regroupe les interactions avec une page.

Un changement de sélecteur est centralisé.

`src/pages/`

World

Objet contenant l'état d'un scénario : page, contexte navigateur, données.

Il évite le partage accidentel d'état entre scénarios.

`src/core/world.ts`

Hooks

Fonctions Before et After exécutées autour des scénarios.

Préparation, preuves, diagnostic et nettoyage.

`src/hooks/hooks.ts`

Assertion

Comparaison entre résultat observé et résultat attendu.

Exemple : vérifier qu'un élément est visible.

`Playwright expect`

À expliquer : « Cucumber trouve la fonction associée à la phrase. Le step utilise un Page Object. Playwright agit sur le navigateur, puis une assertion vérifie le résultat. »

Distinction essentielle : le runner de ce projet UI est **CucumberJS**. Playwright est utilisé pour l'automatisation et les assertions. Ne pas présenter ce projet comme une suite pilotée par le runner Playwright Test.

Où retrouver chaque responsabilité ?

Chemins relatifs au dossier ui_playwright_bdd ; seules les branches utiles à l'architecture sont affichées.

```
ui_playwright_bdd/
├── cucumber.js
├── package.json
├── src/
│   ├── features/
│   ├── steps/
│   ├── pages/
│   ├── core/world.ts
│   ├── hooks/hooks.ts
│   ├── api/QACartApiClient.ts
│   ├── support/
│   ├── config/
│   └── utils/
├── scripts/agents/
│   ├── *-agent.js
│   ├── llm.js
│   ├── jira-fetcher.js
│   └── shared/
├── prompts/
├── qa-knowledge/
├── evaluation/deepeval/
├── config/
└── docs/
```

Tests et préparation API

Les steps et les pages portent le comportement des tests ; le client API prépare des données.

L'application testée est QACart Todo.

Configuration

cucumber.js définit les profils, tags, fichiers chargés et reporters.

Les variables d'environnement choisissent URL, navigateur et fournisseurs.

Artefacts d'exécution

allure-runs/, test-results/, logs/ et memory/ accueillent les résultats et traces selon les commandes.

Ces données sont distinctes du code source.

13 points d’entrée, des rôles différents

Un module nommé « agent » n’utilise pas nécessairement un LLM pour chaque opération.

Pipeline

Orchestrateur : lance les étapes et collecte leurs états.

Pas de LLM propre.

`pipeline-agent.js`

Planning

Analyse couverture, exigences et cas manquants.

Suggestions IA selon la commande.

`planning-agent.js`

Codegen

Propose Gherkin, steps et Page Objects.

Le code reste à contrôler.

`codegen-agent.js`

Runner

Exécute Cucumber et exploite les résultats.

Résumé IA optionnel.

`runner-agent.js`

Allure

Génère et consulte le rapport de tests.

Reporting technique.

`allure-agent.js`

Quality

Triage, causes possibles et cohérence.

Analyse appuyée sur les preuves.

`quality-agent.js`

Bug

Analyse les défauts et propose des corrections.

Réparation : mode distinct.

`bug-agent.js`

Advisor

Recommande GO/NO-GO et applique le gate.

Dépend des preuves et contrôles.

`advisor-agent.js`

Reporting

Produit les dashboards et intégrations.

Envois externes selon la commande.

`reporting-agent.js`

Observability

Suit traces, métriques et coûts.

Calculs principalement déterministes.

`observability-agent.js`

CI

Interagit avec Git et GitHub Actions.

IA pour certains textes.

`ci-agent.js`

Governance

Expose audits et validations de décisions.

Contrôles selon la politique.

`governance-agent.js`

QA Workflow

Suit les étapes de revue et leurs statuts.

Module de pilotage distinct.

`qa-workflow-agent.js`

À retenir : **l’agent porte la tâche QA** ; **le harness fournit l’infrastructure IA** ; **le modèle produit une réponse**.

L’environnement autour du modèle

Harness : ensemble des mécanismes qui rendent l’usage du LLM contrôlable et observable.

Contexte
Instructions et sources pertinentes

→

Appel
Interface commune llm.js

→

Transport
Gateway, quotas et fournisseur

→

Sortie
Contrôles selon l’opération

Instructions

Les prompts précisent le rôle, la tâche et le format attendu.

prompt-store.js + prompts/

Contexte

Le RAG recherche des extraits ; la mémoire conserve des épisodes.

hybrid-rag.js + memory-store.js

Interface LLM

llm.js uniformise l’accès à Groq et Ollama.

Texte, JSON structuré, analyses et comparaisons.

Résilience

Cache et circuit breaker limitent les appels inutiles ou les échecs répétés.

circuit-breaker.js

Vérification

Les sorties sont contrôlées selon leur nature et leur utilisation.

agent-verifier.js, generated-code.js, source-verification.js

Observabilité

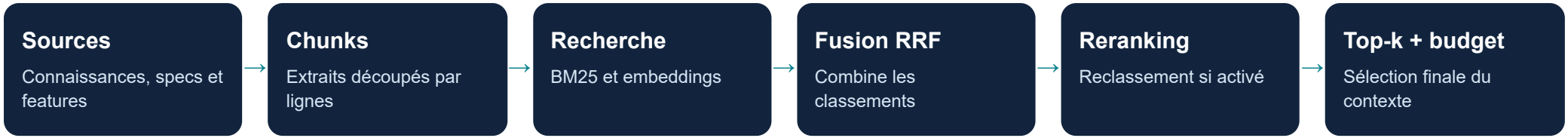
Trace les appels et mesure durées, consommation et coûts estimés.

tracer.js + task-metrics.js

Il n’existe pas un fichier unique « llm-harness ». Cette frontière regroupe plusieurs modules. Les vérificateurs sont appelés par les agents/pipeline concernés, pas systématiquement après chaque appel LLM.

Rechercher avant de répondre

RAG = Retrieval-Augmented Generation : génération enrichie par des informations retrouvées dans un corpus.



BM25

Recherche lexicale : valorise les correspondances de mots.

Utile pour un nom de test ou une erreur exacte.

Embedding

Vecteur numérique représentant le contenu d'un extrait.

La similarité aide à retrouver un sens proche ; embeddings via Ollama.

RRF

Reciprocal Rank Fusion : combine les positions de plusieurs classements.

Associe recherche lexicale et sémantique.

Reranking

Un modèle réévalue la pertinence des extraits candidats.

Optionnel ; peut envoyer les extraits à Groq.

Top-k

Nombre maximal d'extraits à retenir dans le contexte.

4 par défaut ; le budget peut réduire le nombre réellement transmis.

Provenance

Chemin, lignes et empreinte permettent de retrouver les sources.

Les épisodes IA restent non vérifiés ; les vecteurs sont stockés localement.

> Le RAG améliore l'accès au contexte ; il ne garantit pas l'exactitude. La documentation du module exclut notamment les .env et les logs bruts de l'index.

Un document devient des extraits traçables

Chunk = fragment de texte utilisé comme unité de recherche. Ce n’est ni un token, ni un embedding.

Document source
Texte lu depuis le corpus autorisé

→

Découpage
Regroupe les lignes entières

→

Chunks
Fragments de 1 800 caractères maximum

→

Index
Termes BM25 et vecteurs associés

Pourquoi découper ?

Retrouver un passage pertinent sans transmettre un document complet au modèle.

Un même document peut fournir plusieurs chunks.

Comment dans ce code ?

chunkFile() accumule les lignes jusqu’à la limite de 1 800 caractères.

Pas de chevauchement explicite. Une ligne trop longue est omise et signalée.

`scripts/agents/shared/rag.js`

Que conserve un chunk ?

Texte, identifiant, chemin, lignes de début et de fin, empreinte de la source et type.

Ces métadonnées permettent de retrouver et vérifier l’extrait.

Chunk et embedding

Le chunk est du texte lisible ; l’embedding est sa représentation numérique.

Le vecteur aide à chercher. Le texte retenu constitue le contexte du LLM.

Taille et compromis

Un fragment court cible mieux un passage, mais peut perdre du contexte.

Un fragment long contient plus de contexte, mais peut ajouter du bruit.

Cas de la mémoire IA

Le découpage traite séparément les lignes d’épisodes sélectionnés.

L’historique reste identifié comme non vérifié, même après indexation.

À dire au client : « Je découpe les sources en passages identifiables. Le RAG retrouve ces passages et conserve leur provenance pour permettre la vérification. »

Combien de chunks envoyer au modèle ?

Top-k = plafond du nombre d’extraits sélectionnés après classement. Le budget de contexte s’applique aussi.

Chunks candidats
Résultats BM25 et sémantiques

→

Classement
Fusion RRF puis reranking si activé

→

Sélection
Au plus k chunks qui tiennent dans le budget

→

LLM
Reçoit les textes et leurs références

Exemple fictif : k = 4

C1	Règle métier de connexion	RETENU
C2	Scénario de connexion existant	RETENU
C3	Précondition du compte de test	RETENU
C4	Note QA sur le message attendu	RETENU
C5	Ancienne analyse peu pertinente	NON RETENU
C6	Règle de suppression sans rapport	NON RETENU

Illustration uniquement : ces six extraits ne proviennent pas d’une recherche exécutée. On suppose que les quatre premiers tiennent dans le budget.

Valeurs par défaut du code

topK = 4 et maxChars = 4 500.

Le coût utilisé est la longueur du chunk sérialisé en JSON, métadonnées comprises. Ce ne sont pas 4 500 tokens.

`rag.js : selectHits() ; hybrid-rag.js : search()`

Pourquoi moins de quatre ?

Le corpus peut être insuffisant ou le budget ne pas permettre de retenir quatre chunks.

Un extrait qui dépasse le budget restant est sauté ; la sélection peut en retenir un suivant plus petit.

Ne pas confondre

k est le nombre maximal de chunks, pas le nombre de documents ou de réponses du LLM.

La limite de candidats du reranker est une étape distincte du top-k final.

À dire au client : « Je classe les chunks, puis je sélectionne au maximum quatre extraits dans le budget prévu. Augmenter k peut enrichir le contexte, mais aussi ajouter du bruit et de la consommation. »

Trois questions à séparer

Peut-on envoyer la requête ? Peut-on utiliser la réponse ? Peut-on prendre la décision ?

1 · Gateway IA

Contrôle les destinations, modèles, secrets reconnus, tailles, délais et quotas.

ai-gateway.js + config/ai-gateway.json

2 · Vérificateurs

Contrôlent format, code, références et contraintes selon la sortie.

Un JSON valide ne démontre pas une conclusion métier.

3 · Gouvernance

Suit les validations et les décisions nécessitant une intervention humaine.

governance-agent.js + ai-governance.js

Le workflow de revue disponible

Gherkin — Valider le comportement décrit → **Script** — Relire l'implémentation → **Exécution** — Contrôler les preuves → **Analyse** — Examiner les conclusions → **Décision** — Approuver ou bloquer

Le module qa-workflow existe séparément. Il ne faut pas affirmer que toutes ces revues sont automatiquement imposées par chaque commande du pipeline. Le gateway est applicatif : ce n'est ni un pare-feu ni une garantie contre toutes les fuites ou injections.

Ce que fait réellement le mode full

Une séquence définie dans pipeline-agent.js ; ce n’est pas un agent qui choisit librement toutes ses actions.

01 · Planning
Analyse de couverture.

02 · Codegen
Génération des spécifications.

03 · Runner
Exécution des tests.

04 · Allure
Production du rapport.

05 · Quality
Triage et analyse complète.

06 · Bug
Analyse des échecs.

07 · Reporting
Dashboard KPI.

08 · Jira
Synchronisation si --sync-jira.

09 · Advisor
Recommandation de livraison.

10 · Observability
Métriques du traitement.

11 · CI
Consultation de l’état GitHub.

12 · Gate
Contrôle final de validation.

quick

Smoke → Allure → triage → dashboard → gate.

Un parcours plus court.

report

Exploite les résultats existants pour produire des rapports.

Ne valide pas une nouvelle exécution.

Limite du mode full

Il appelle codegen spec et bug analyze.

Il ne génère pas systématiquement tous les scripts et ne répare pas automatiquement les tests.

Le contrôle final tient compte de l’exécution complète, des résultats du run et des étapes obligatoires. Une recommandation IA ne remplace pas ces conditions.

Mesurer les tests et mesurer l’IA

Deux objets différents : le comportement de l’application et la qualité des réponses des modèles.

Allure

Rapport de tests avec statuts et pièces jointes.

Un rapport peut être partiel ; sa présence ne signifie pas réussite.

Traces et captures

Éléments pour reconstituer un échec navigateur.

Les hooks collectent les informations selon les conditions du run.

Manifeste de run

Identifie l’exécution et conserve sa complétude et son code de sortie.

Évite d’utiliser de vieux résultats comme preuve nouvelle.

Métriques IA

Durée, consommation et coûts estimés des appels.

Un tarif inconnu ne doit pas devenir un coût nul.

DeepEval

Évalue les réponses IA sur un jeu de cas et leur contexte.

Lancé à la demande, dans un environnement Python séparé.

Fidélité / pertinence

Fidélité : appui sur le contexte. Pertinence : réponse à la question.

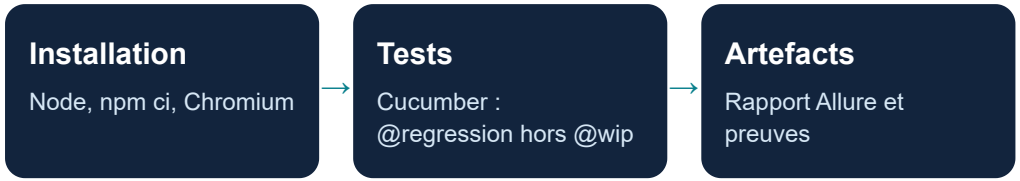
Ces scores ne prouvent pas la conformité de l’application.

DeepEval ne juge pas automatiquement chaque réponse de chaque agent. Ses résultats dépendent du jeu de référence, du contexte fourni et du juge.

Deux chemins d’exécution

Le workflow GitHub existant exécute les tests ; le pipeline d’agents est une commande distincte.

GitHub Actions



Déclenchements présents : push, pull request, lancement manuel et planification. Le fichier de référence est `.github/workflows/ci-playwright.yml`, à la racine du dépôt.

Commandes à expliquer

```
# Tests UI
npm run test:headless

# Pipeline court / complet
npm run agent:pipeline:quick
npm run agent:pipeline:full

# Recherche et métriques
npm run rag:status
npm run metrics:report

# Évaluation IA à la demande
npm run eval:deepeval
```

> Ces commandes sont des repères de préparation, pas la preuve qu’elles ont été exécutées ici. Certaines nécessitent les accès Jira, au fournisseur LLM et à l’application. Le workflow CI consulté ne lance pas l’intégralité des agents IA.

Les questions probables du client

Dans la version HTML, cliquez sur une question pour révéler la réponse. Dans le PDF, toutes les réponses sont visibles.

Pourquoi Cucumber et Playwright ?

Cucumber relie les scénarios métier aux steps. Playwright exécute les actions navigateur et fournit les assertions.

Agent, modèle et harness : quelle différence ?

L'agent porte une tâche QA. Le modèle génère une réponse. Le harness prépare, encadre et observe l'utilisation du modèle.

Comment vérifiez-vous le code généré ?

Par des contrôles de syntaxe, de types selon le chemin utilisé, une revue métier et une exécution. Aucun de ces contrôles seul ne suffit.

Le RAG évite-t-il les hallucinations ?

Il fournit des sources pertinentes et traçables, mais une réponse peut encore être fausse. Je vérifie ses références et ses conclusions.

Qui décide du GO / NO-GO ?

Advisor formule une recommandation et un gate. La décision doit respecter les preuves, les blocages techniques et la gouvernance humaine applicable.

Le framework répare-t-il tout seul ?

Un mode de réparation existe, mais le pipeline full consulté lance bug analyze. Je ne promets pas une réparation automatique de tous les échecs.

Que se passe-t-il si le LLM est indisponible ?

Les mécanismes de résilience et les erreurs sont gérés selon le chemin d'appel. Le Runner peut lancer Cucumber sans attendre un résumé IA.

Tout est-il branché à la CI ?

La CI UI lance Cucumber/Playwright et Allure. Le pipeline complet des agents constitue actuellement un autre chemin.

> Si un détail vous échappe : « Je préfère vérifier le chemin d'exécution dans le code. Je peux vous montrer le module qui le gère. »

Un pitch de 60 secondes

L’objectif : expliquer le parcours d’un test et situer l’IA avec précision.

« Mon framework UI repose sur Cucumber, Playwright et TypeScript. Les features décrivent les comportements métier ; les steps s’appuient sur des Page Objects et un contexte isolé par scénario. Les assertions et les preuves Allure permettent d’analyser le résultat.

J’ai ajouté une couche d’agents pour la préparation, l’analyse des échecs et les recommandations. Leur LLM harness rassemble le contexte documentaire, les appels aux modèles, les contrôles, la résilience et les métriques.

Je distingue toujours une proposition IA d’un résultat démontré : je vérifie les sources, le code et les tests, et je conserve les validations QA nécessaires. »

À montrer en premier

Une feature, son step, son Page Object et l’assertion associée.

Expliquez ce parcours avant les agents.

À montrer ensuite

Un rapport avec les preuves, puis le rôle du harness.

Partez d’un exemple concret d’échec.

À éviter

Promettre une IA autonome et infaillible ou une couverture totale.

Distinguez code présent, configuration et exécution prouvée.

Retrouver, vérifier, expliquer

Le code des commandes prévaut sur les anciens résumés du README lorsque les descriptions diffèrent.

BDD

`cucumber.js ; src/core/world.ts ; src/hooks/hooks.ts ; src/pages/BasePage.ts`

Orchestration

`scripts/agents/pipeline-agent.js ; shared/pipeline-runtime.js ; docs/pipeline-execution.md`

LLM et harness

`scripts/agents/llm.js ; scripts/agents/shared/ ; prompts/`

RAG

`docs/rag.md ; scripts/agents/shared/hybrid-rag.js ; reranker.js`

Gateway

`docs/ai-gateway.md ; config/ai-gateway.json ; scripts/agents/shared/ai-gateway.js`

Vérification

`docs/agent-verification.md ; scripts/agents/shared/agent-verifier.js`

Gouvernance

`scripts/agents/governance-agent.js ; qa-workflow-agent.js ; shared/qa-workflow-engine.js`

Évaluation

`docs/deepeval.md ; evaluation/deepeval/ ; scripts/deepeval.js`

CI

`.github/workflows/ci-playwright.yml à la racine du dépôt QA_Plateforme`

Oracle

Résultat de référence permettant de juger un test.

Flaky

Test dont le résultat varie sans changement fonctionnel pertinent.

RCA

Root Cause Analysis : recherche de la cause d’un échec.

Quality gate

Conditions à satisfaire pour autoriser la suite du processus.

Circuit breaker

Mécanisme de protection face aux échecs répétés d’un service.

Token

Unité utilisée par un modèle pour traiter et produire du texte.

> Support pédagogique construit à partir du dépôt local. Aucun appel Jira, fournisseur LLM ou pipeline de production n’a été nécessaire à sa création. Le PDF est une version fixe ; l’animation et les contrôles sont dans le HTML compagnon.